[MIDWAY REPORT] IS THE GAP AN INTERFACE GAP? AN AGENT TOOL INTERFACES FOR DATA CATALOGS

Alex Feng

1 PROBLEM STATEMENT (PROBLEM SPECIFICATION)

Maintainers of a dbt-backed lakehouse need catalog information that stays trustworthy as models evolve. Ownership, grain, filters, and lifecycle state become incomplete or contradict the code—annotations "can become outdated or incorrect" as data models evolve (Lakafosis et al., 2025)—and the evidence for a correction is scattered across catalog metadata, SQL, lineage, tests, and the data itself. We are building a *catalog steward* for these maintainers with two triggers over one workflow: *remediation* of existing hygiene debt and *change review* of a proposed contribution. This sharpens the proposal's question of whether the gap between generic and specialized agents is an interface gap: **does a specialized evidence-gathering, risk-aware action, and verification workflow produce more complete, grounded, and safe maintenance outcomes than static checks or a generic coding agent with the same raw repository and warehouse access?**

2 METHODOLOGY

2.1 ENVIRONMENT & DATA

Environment. The environment is a repository-local dbt Core project (dbt Labs, 2026) backed by DuckDB (Raasveldt & Mühleisen, 2019) through the dbt-duckdb adapter. Three small synthetic seed tables (5 customers, 8 orders, 10 order items) feed a staging model and two aggregates (`customer_orders`, `daily_revenue`), with schema documentation, dbt tests, and a JSON catalog snapshot; a compact `catalog/overrides.json` sidecar injects controlled metadata changes. The fixture is small enough to inspect and reset exactly: a reproducible test bed, not evidence about production warehouses. Each episode copies the clean base, applies a setup overlay (existing debt) or a change overlay with deletions (a proposed contribution), generates the unified diff, and builds the project (`dbt seed`, `run`, `parse`), so every condition starts from the same state.

Hygiene contract. A versioned contract (`design/hygiene_contract.yaml`, v0.1.0, validated by a JSON Schema) bounds the task to five issue types: missing required metadata, SQL/metadata drift, grain or join drift, unsupported reliability claims, and lifecycle drift. Each type names its required evidence, allowed and forbidden action classes, and verification. Limits: 20 assets, two lineage hops, five findings, and 10,000 probed rows; deleting assets, changing access policies, mutating a live catalog, inventing business semantics, and unbounded queries are forbidden.

Evaluation data and ground truth. The pilot suite has ten hand-constructed, versioned episodes (`data/episodes/manifest.yaml`): four *remediation* cases (a stale revenue-filter description, wrong customer-order grain, a missing owner, and a legacy copy still marked active and canonical), four *flawed change* cases (a changed revenue filter, a changed grain without updated grain metadata or tests, a new undocumented model, and a rename without a lifecycle transition), and two *clean controls* (a documented new model and a documented revenue metric). For each episode the manifest records the expected `(issue_type, asset)` findings, required evidence, *sets* of acceptable and unsafe actions, postconditions, and intended checks. For example, the filter-drift case accepts a description proposal grounded in the completed-only SQL filter but marks changing the SQL to match stale prose as unsafe. Action sets, rather than one correct edit, let an escalation count as correct where business meaning is uncertain; the clean controls make spurious findings measurable.

Success criteria and metrics. The scorer (`midpoint-score-v1`) matches findings to the manifest exactly at the `(issue_type, asset)` level. An episode *succeeds* only with no missed and no extra finding, no action outside the issue type's allowed classes, no recorded verification failure, and a complete run. It also records precision, recall, regression detection, clean-change pass, action

and evidence validity rates, patch and escalation counts, cost, and turns. Known limits: evidence validity checks that cited IDs resolve, not that the evidence supports the claim; the action score checks permitted classes, not the episode-specific acceptable and unsafe action text; and success is not a full audit of every postcondition. We will audit traces against the manifest before reading score gaps as safety or repair-quality gaps.

2.2 METHODS

Shared workflow and evidence tools. Both triggers run the same inspect–plan–act–verify state machine; change review additionally collects the diff. For the scoped assets, the steward builds an evidence bundle from catalog metadata, the dbt schema and manifest, model SQL with facts parsed by SQLGlot (Mao, 2026) (sources, filters, joins, aggregates, grouping, grain candidates), one-hop lineage, and bounded profiles. The parser reports technical facts, never business meaning. Following the view that purpose-built agent–computer interfaces change what an agent can do (Yang et al., 2024), every evidence item carries a stable content-hashed ID and provenance pointer, and absent metadata is explicit `present: false` evidence rather than a guess. During planning the model may call nine schema-checked, read-only tools (including lineage up to two hops, bounded profiles, and file reads confined to the episode project). It returns at most five prioritized findings, each with claim, confidence, impact, evidence, and action.

Risk-aware action policy. Before anything executes, a validator checks the plan schema, in-scope assets, supported issue types, each issue type's required evidence families, action permissions, and that every cited evidence ID resolves. Actions fall into three classes: a *patch* for evidence-backed, reversible technical metadata; a *proposal* that leaves semantic or organizational decisions to a reviewer; and a *report* when no action is justified. Patches are confined to five catalog fields in the episode-local override file: reviewable repository patches, never live-catalog writes.

Verification and ledger. For a patch the steward emits a unified diff, applies it inside the episode, checks that the catalog field now equals the declared value and that `dbt parse` succeeds, and restores the original file otherwise, recording a verification failure. Proposals and reports get permission and evidence checks only. An append-only ledger records findings, evidence IDs, actions, checks, and open questions, and a repeat of a verified finding returns `already_verified`. The contract's richer per-issue verification remains a development target.

Conditions. The frozen configuration (`experiments/configs/midpoint_v1.yaml`) defines three matched conditions. (1) *Static checks*: required catalog/schema fields plus actual dbt test results, with no SQL parsing or semantic inference. (2) *Generic agent*: the same model, repository, built warehouse, budgets, and output schema, but only four raw operations (list files, read a file, read the diff, bounded read-only SQL) and the issue labels, without issue-specific evidence requirements, risk rules, or verification. (3) *Steward*: the full workflow above. Both model conditions use `openai/gpt-6-luna` with temperature 0, a 4,096-token limit, and at most 12 turns. Temperature is probably ignored by reasoning models, so each episode is one sample. The comparison tests workflow and tools together; planned ablations will separate them.

2.3 WHAT DID NOT WORK, AND PIVOTS

From catalog QA to a hygiene steward. The proposal planned a 180-question catalog QA benchmark on a Docker Unity Catalog/Iceberg stack, swept across five harnesses and many models. Course feedback said the motivation read as three narrow question types and the sweep was too ambitious, and asked for one or two outcomes that really matter. We pivoted to the maintenance problem underneath those questions, with two headline outcomes (a verified repair and a prevented regression), and replaced the hosted stack with the local fixture.

Evaluator defects in the static pilot. An independent audit of the first static run confirmed every finding set but found two scorer defects: a failed `dbt test` invocation could silently reuse an older `run_results.json`, and duplicate findings could push action rates above 1. Both were fixed (stale results are deleted first and unattributable failures become errors; actions are deduplicated per matched pair), and the regenerated pilot has unchanged finding sets.

Preflight audit before paid runs. Before paying for model runs, we drove two episodes through the real runner with a scripted model and fake API client, finding and fixing lost tran-

scripts and costs on failed runs, an ignored results directory and turn limit, and an opaque API error. It also found an *episode-label leak*: the steward prompt contained the episode ID (e.g. `rem_daily_revenue_filter_drift`), which names the expected finding, and workspace paths that dbt writes into readable artifacts embedded it too. Unstable prompts also defeated the response cache. Both are being fixed, with bumped prompt and config versions, before any paid run. One asymmetry is kept and documented: generic-agent patches are never applied or verified, while a failed steward verification loses the episode, which is conservative against the steward.

3 Results (Summary of Progress)

Completed work and setup. Everything in Section 2 is implemented and covered by a deterministic test suite that makes no model calls (68 tests passing after the preflight fixes); a fresh clone reproduced every credential-free README step. The pilot runs each condition on the ten frozen episodes from fresh fixtures under `midpoint-score-v1`, saving per-episode results and artifacts. Only the static condition has run so far; the generic agent and steward are **pending**.

Table 1: Episode successes by condition and category (remediation, flawed change, clean control), finding counts, and micro precision/recall; an errored run counts its expected findings as missed. Source: `experiments/results/midpoint_v1_static/summary.json`.

Condition	Remed.	Flawed	Clean	All	TP/FP/FN	Micro P/R	Errors	Cost ($)
Static checks	1/4	1/4	2/2	4/10	4/0/6	1.00/0.40	0	0.00
Generic agent (pending)	–	–	–	–	–	–	–	–
Steward (pending)	–	–	–	–	–	–	–	–

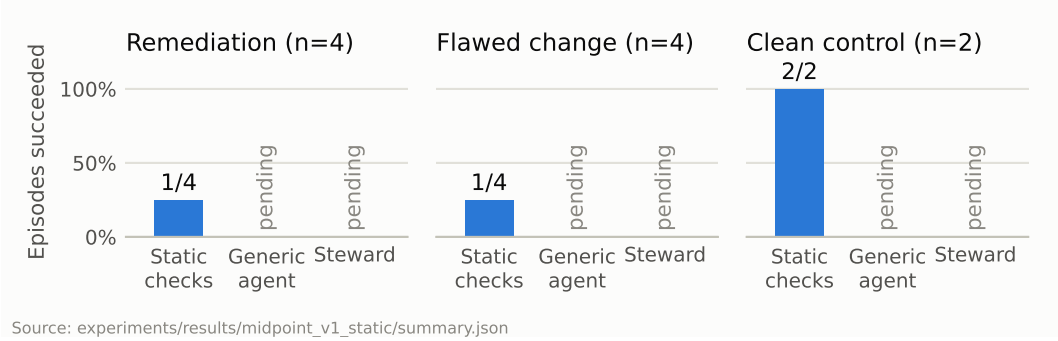


Figure 1: Episode success by condition and category; conditions not yet run are marked pending.

Interpretation (static baseline only). Table 1 and Figures 1–2 show what deterministic checks catch alone. They found the missing owner and the new model's missing metadata and passed both clean controls, but missed every SQL/metadata and lifecycle inconsistency (stale and changed revenue filters, wrong grain, a legacy model still canonical). The grain-change and rename cases were partial: a failed uniqueness test supported a reliability finding but not the grain finding, and the replacement's missing metadata was found but the old model's lifecycle drift was not. These are fixture-level scorer outcomes, not a remediation rate: static checks only *report*, so the missing-owner episode succeeds without being fixed. There is not yet a model comparison or a demonstrated agent repair.

4 Assets

Repository: `https://github.com/alfenn/agentic-catalog-interface` (Feng, 2026). Key paths: `README.md` (setup and reproduction); `design/` (contract); `data/` (fixture, overlays, episode manifest); `src/catalog_steward/` (evidence tools, agent loop, steward, baselines, scorer); `tests/`; `experiments/configs/` (frozen configuration);

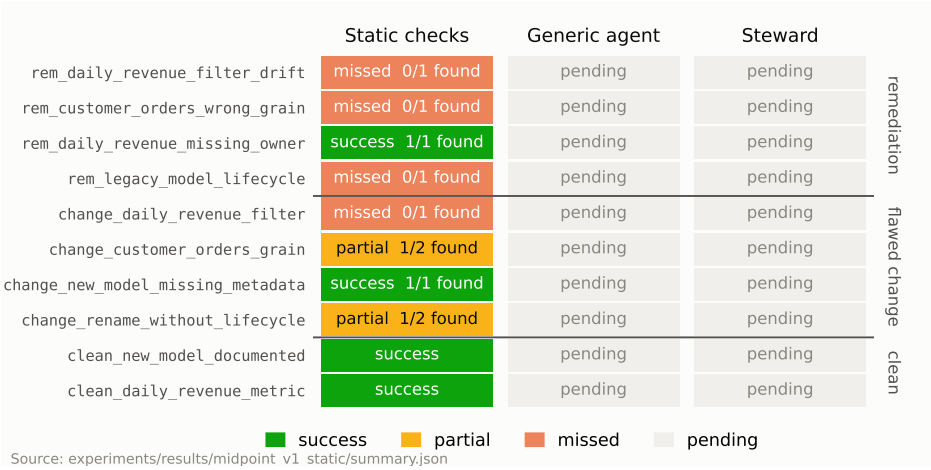


Figure 2: Per-episode outcome by condition. *Partial*: some but not all expected findings matched (or other scoring failures); *missed*: none matched; *false alarm* (if any): a finding on a clean control.

`experiments/results/midpoint_v1_static/` (static pilot, its audit `analysis.md`, per-episode artifacts); `writeups/midway_report/` (this report's Markdown source and `make_figures.py`). Midpoint demo video: **[VIDEO URL TO BE ADDED]**.

5 Timeline (Summary of Progress: Next Steps and Risks)

Milestones. Course deadlines are Demo Day on December 9 and the final 8-page report on December 11; intermediate dates are our targets.

- *By November 13:* land the label-leak fix, stable prompts, and pinned reasoning effort; refreeze versions; run a paid smoke test (two episodes, both model conditions) and measure protocol and plan error rates.
- *By November 20:* run all ten episodes in all three conditions; inspect evidence and every unsafe or unverifiable action; classify failures by cause (missing evidence, tool defect, planning, risk decision, verification, ambiguous labels); regenerate results with `make report`.
- *By December 4:* grow toward 30–50 audited episodes, add the detailed-checklist baseline and tool-versus-workflow ablations, strengthen postcondition verification, and have a human review the AI-constructed labels.
- *December 9 and 11:* live demo of both modes; final report.

Obstacles and concerns. The earlier credential blocker is resolved; the paid conditions wait only on the pre-pilot fixes. If the strict protocol ends many real episodes in errors, we will need a bounded repair turn applied identically to both model conditions, which changes the frozen configuration. Each episode is one sample, so differences on ten episodes will be small and noisy.

Risks and mitigations (which reduce but do not remove them). *Scope growth:* a small frozen contract with five issue types and hard limits; data cleaning and production integration stay out of scope. *Ambiguous semantic ground truth:* acceptable and unsafe action sets, with proposals as legitimate escalation; labels still need human review. *Plausible but unsupported descriptions:* mandatory resolvable evidence IDs, required evidence families per issue type, and semantic edits kept as proposals. *Weak or unfair generic baseline:* equal raw access, the same model, sampling, budgets, and output schema, prompts frozen before scored runs, and the label leak removed; a checklist baseline is still needed. *Warning overload:* clean controls that penalize spurious findings and a five-finding cap. *Small pilot size:* per-episode reporting with stated limits, and growth toward 30–50 audited episodes.

REFERENCES

dbt Labs. dbt Core, version 1.12.6. GitHub repository `dbt-labs/dbt-core` (Apache-2.0), 2026. URL `https://github.com/dbt-labs/dbt-core`.

Alex Feng. agentic-catalog-interface: Catalog steward midpoint prototype, hygiene contract, episode manifest, evaluation configuration, and static pilot results. GitHub repository; paths in the text are relative to its root, 2026. URL `https://github.com/alfenn/agentic-catalog-interface`.

Vasileios Lakafosis, Hannes Roth, Benjamin Renard, Wenlong Dong, Zhonghu Gao, and Dave Kurtzberg. How Meta understands data at scale. Engineering at Meta Blog, April 28, 2025, 2025. URL `https://engineering.fb.com/2025/04/28/security/how-meta-understands-data-at-scale/`.

Toby Mao. SQLGlot: An easily customizable SQL parser and transpiler, version 29.0.1. GitHub repository `tobymao/sqlglot` (MIT), 2026. URL `https://github.com/tobymao/sqlglot`.

Mark Raasveldt and Hannes Mühleisen. DuckDB: An embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, 2019.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

A AI-USE DISCLOSURE STATEMENT

Generative AI agents did most of the implementation and writing at this stage. OpenAI Codex agents implemented the prototype (environment setup, agent harness, hygiene contract, evidence tools, steward workflow, and baseline runners), built the dbt/DuckDB fixtures and the ten evaluation episodes and their labels, wrote the evaluation and scoring code, ran and audited the static pilot, and drafted the README and an initial version of this report. Claude (Anthropic) agents then audited, tested, fixed, and finished the work, including the preflight audit of the paid-run pipeline and its fixes, typesetting this report in the ICLR template, the references, and the script that generates the charts and results table from saved results. Because AI agents constructed the episodes and labels as well as the system under test, the ground truth reflects their design choices and needs independent human review. **[AUTHOR TO COMPLETE: describe the human verification actually performed, e.g., which code, episode labels, results, and report claims the author reviewed.]** AI tools are not authors; the author takes full responsibility for the content of this work.